

الفصل الأول: وصف المشروع

1.1 مقدمة المشروع

مع التطور المتسارع في التحول الرقمي أصبحت إدارة الفعاليات والمؤتمرات والمهرجانات تتطلب أنظمة إلكترونية متطورة تساعد في تنظيم عمليات الحجز وإدارة التذاكر ومتابعة الحضور بشكل أكثر كفاءة. وتعتمد العديد من الجهات المنظمة للفعاليات على أساليب تقليدية تؤدي إلى صعوبة إدارة الحجوزات وتتبع التذاكر والتحقق من صلاحيتها. لذلك تم تطوير نظام **Eventify** كنظام إلكتروني متكامل لإدارة الفعاليات وحجز التذاكر، حيث يوفر بيئة موحدة تجمع بين منظمي الفعاليات والحضور وإدارة النظام ضمن منصة واحدة سهلة الاستخدام.

1.2 فكرة المشروع

يقوم المشروع على إنشاء منصة إلكترونية تتيح لمنظمي الفعاليات إنشاء وإدارة فعالياتهم، بينما يستطيع المستخدمون استعراض الفعاليات المتاحة وحجز التذاكر إلكترونياً وإدارتها من خلال حساباتهم الشخصية. كما يوفر النظام لوحة تحكم خاصة بالمدير لمتابعة أداء المنصة واعتماد الفعاليات ومراقبة الأنشطة المختلفة داخل النظام.

1.3 أهداف المشروع

يهدف المشروع إلى:

- تسهيل عملية إدارة الفعاليات إلكترونياً .
- أتمتة عمليات حجز وشراء التذاكر .
- تقليل الأخطاء الناتجة عن الإدارة اليدوية .
- تحسين تجربة المستخدم أثناء البحث عن الفعاليات وحجز التذاكر .
- توفير وسيلة آمنة للتحقق من صحة التذاكر باستخدام QR Code .
- توفير لوحة تحكم لمتابعة أداء النظام وإدارة الفعاليات .

1.4 الفئات المستهدفة

يستهدف النظام ثلاث فئات رئيسية:

الحضور (Attendees)

الأشخاص الراغبون في استعراض الفعاليات وشراء التذاكر.

منظمو الفعاليات (Organizers)

الجهات أو الأفراد المسؤولون عن إنشاء وإدارة الفعاليات.

مدير النظام (Admin)

المسؤول عن إدارة المنصة واعتماد الفعاليات ومتابعة العمليات المختلفة.

1.5 المزايا الرئيسية للنظام

يوفر النظام مجموعة من المزايا أهمها:

- تسجيل المستخدمين وتسجيل الدخول .
- إدارة الصلاحيات حسب نوع المستخدم .
- إنشاء وإدارة الفعاليات .
- إدارة أنواع التذاكر وأسعارها .
- شراء التذاكر إلكترونياً .
- إنشاء تذاكر تحتوي على QR Code .
- تحميل التذاكر بصيغة PDF .
- إدارة الملف الشخصي للمستخدم .
- نظام إشعارات للمستخدمين .
- لوحة تحكم إحصائية للمسؤولين .

1.6 التقنيات المستخدمة

الواجهة الخلفية (Backend)

- Laravel Framework
- Laravel Sanctum Authentication
- MySQL Database
- DomPDF
- Simple QR Code

الواجهة الأمامية (Frontend)

- Vue.js 3
- Vue Router
- Axios
- Tailwind CSS

الفصل الثاني: خطة المشروع

2.1 منهجية تطوير المشروع

تم تطوير نظام Eventify باستخدام منهجية التطوير التكراري (Iterative Development) ، حيث تم تقسيم المشروع إلى مراحل متتابعة تبدأ بتحليل المتطلبات ثم التصميم والتطوير والاختبار وصولاً إلى النسخة النهائية للنظام. ساعد هذا الأسلوب في بناء النظام بصورة تدريجية مع إمكانية مراجعة الوظائف وإجراء التحسينات بشكل مستمر أثناء عملية التطوير.

2.2 مرحلة جمع وتحليل المتطلبات

في هذه المرحلة تم تحديد احتياجات المستخدمين والفئات المستهدفة للنظام، وتم تقسيم المتطلبات إلى ثلاثة أنواع من المستخدمين:

الحضور (Attendees)

- إنشاء حساب جديد .
- تسجيل الدخول للنظام .
- استعراض الفعاليات المتاحة .
- حجز وشراء التذاكر .
- عرض التذاكر الخاصة بهم .
- تحميل التذاكر بصيغة PDF .
- عرض رمز QR الخاص بالتذكرة .

منظمو الفعاليات (Organizers)

- إنشاء الفعاليات .
- تعديل بيانات الفعاليات .
- حذف الفعاليات .
- إدارة أنواع التذاكر وأسعارها .
- متابعة الحضور والتذاكر المباعة .
- التحقق من التذاكر باستخدام QR Code .

مدير النظام (Admin)

- مراجعة الفعاليات .
- قبول أو رفض الفعاليات .
- متابعة إحصائيات النظام .
- مراقبة الأنشطة المختلفة داخل المنصة .

2.3 مرحلة تصميم النظام

تم تصميم النظام وفق بنية Client-Server لفصل الواجهة الأمامية عن الواجهة الخلفية.

الواجهة الأمامية (Frontend)

تم تصميم واجهات الاستخدام باستخدام Vue.js بحيث تكون:

- سهولة الاستخدام .
- متجاوبة مع مختلف أحجام الشاشات .
- تدعم الوضع الليلي (Dark Mode).
- توفر تجربة استخدام سلسة للمستخدم .

الواجهة الخلفية (Backend)

تم بناء واجهات البرمجة (APIs) باستخدام Laravel لتوفير:

- إدارة المستخدمين .
- إدارة الفعاليات .
- إدارة التذاكر .
- إدارة الطلبات .
- إدارة الإشعارات .
- التحقق من الصلاحيات .

2.4 مرحلة بناء قاعدة البيانات

تم تصميم قاعدة البيانات لتخزين وإدارة بيانات النظام بطريقة مترابطة ومنظمة.

وتشمل أهم الكيانات:

- المستخدمون (Users)
- الفعاليات (Events)
- أنواع التذاكر (Ticket Types)
- الطلبات (Orders)
- عناصر الطلب (Order Items)
- التذاكر (Tickets)
- الإشعارات (Notifications)

كما تم ربط الجداول باستخدام العلاقات المناسبة لضمان تكامل البيانات وسهولة الوصول إليها.

2.5 مرحلة التطوير البرمجي

تم تنفيذ المشروع على جزأين رئيسيين:

تطوير الواجهة الخلفية

تم تطوير مجموعة من الـ APIs المسؤولة عن:

- المصادقة وتسجيل الدخول .
- إدارة الفعاليات .
- إدارة الطلبات .
- إدارة التذاكر .
- إدارة الملف الشخصي .
- إدارة الإشعارات .
- إدارة لوحة التحكم .

تطوير الواجهة الأمامية

تم إنشاء صفحات رئيسية تشمل:

- صفحة تسجيل الدخول وإنشاء الحساب .
- صفحة استعراض الفعاليات .
- صفحة إدارة الفعاليات .
- لوحة التحكم .
- صفحة التذاكر الخاصة بالمستخدم .
- صفحة الملف الشخصي .

2.6 مرحلة الاختبار

تم اختبار النظام للتحقق من:

- صحة تسجيل الدخول وإنشاء الحساب .
- صلاحيات المستخدمين المختلفة .
- إنشاء الفعاليات وتعديلها وحذفها .
- عمليات شراء التذاكر .
- إنشاء ملفات PDF الخاصة بالتذاكر .
- توليد أكواد QR والتحقق منها .
- استقرار واجهات البرمجة API.

كما تم اختبار تجربة المستخدم للتأكد من سهولة الاستخدام وسرعة تنفيذ العمليات المختلفة.

2.7 مخرجات المشروع

تمثلت المخرجات النهائية للمشروع في:

- منصة إلكترونية متكاملة لإدارة الفعاليات .
- نظام حجز وشراء تذاكر إلكتروني .
- نظام تحقق باستخدام QR Code.

- لوحة تحكم إدارية لإدارة المنصة .
- واجهات استخدام حديثة ومتجاوبة .
- واجهات API قابلة للتطوير والتوسع مستقبلاً.

الفصل الثالث: المشكلة التي يعالجها المشروع

3.1 مقدمة

تشهد الفعاليات والمؤتمرات والمعارض في الوقت الحالي نمواً متزايداً، الأمر الذي يتطلب وجود أنظمة إلكترونية متطورة لإدارة عمليات التسجيل والحجز ومتابعة الحضور. ومع ذلك ما زالت العديد من الجهات تعتمد على إجراءات تقليدية أو أنظمة متفرقة تؤدي إلى انخفاض الكفاءة التشغيلية وصعوبة إدارة الفعاليات بصورة فعالة.

ومن هنا جاءت الحاجة إلى تطوير نظام متكامل يربط جميع الأطراف المشاركة في الفعالية ضمن منصة واحدة تتيح إدارة دورة حياة الفعالية بشكل إلكتروني ابتداءً من إنشائها وحتى حضور المشاركين لها.

3.2 المشكلة الحالية

تعاني عمليات إدارة الفعاليات التقليدية من مجموعة من التحديات والمشكلات، من أهمها:

أولاً: صعوبة إدارة الفعاليات

يعتمد العديد من منظمي الفعاليات على وسائل يدوية أو أدوات متعددة غير مترابطة لإدارة فعالياتهم، مما يؤدي إلى صعوبة متابعة المعلومات وتحديثها بشكل مستمر.

ثانياً: ضعف إدارة التذاكر

تواجه الجهات المنظمة مشكلات في تتبع التذاكر المباعة ومعرفة عدد المقاعد المتبقية والتحقق من صلاحية التذاكر عند دخول الحضور.

ثالثاً: بطء إجراءات الحجز

تتطلب بعض الأنظمة التقليدية إجراءات طويلة للحجز والتسجيل، مما يؤثر سلباً على تجربة المستخدم ويقلل من كفاءة عملية شراء التذاكر.

رابعاً: غياب المتابعة والإحصائيات

لا تتوفر في كثير من الأنظمة التقليدية أدوات تحليلية تساعد الإدارة على متابعة أداء الفعاليات أو قياس الإيرادات وعدد الحضور بشكل دقيق.

خامساً: ضعف التحقق من صحة التذاكر

قد يؤدي الاعتماد على التذاكر الورقية أو الطرق اليدوية إلى زيادة احتمالية التزوير أو استخدام التذكرة أكثر من مرة.

3.3 آثار المشكلة

تؤثر هذه المشكلات على جميع الأطراف المرتبطة بالفعالية، حيث ينتج عنها:

- زيادة الوقت والجهد المطلوب لإدارة الفعاليات .
- ارتفاع نسبة الأخطاء البشرية .
- ضعف تجربة المستخدم أثناء الحجز .
- صعوبة مراقبة الإيرادات والمبيعات .
- تأخير إجراءات الدخول إلى الفعاليات .
- انخفاض مستوى التنظيم والكفاءة التشغيلية .

3.4 الحل المقترح

يقدم نظام **Eventify** حلاً إلكترونياً متكاملًا لمعالجة هذه المشكلات من خلال:

- توفير منصة موحدة لإدارة الفعاليات .
- أتمتة عمليات الحجز وإصدار التذاكر .
- إنشاء تذاكر إلكترونية تحتوي على QR Code .
- توفير لوحة تحكم لمتابعة الأداء والإحصائيات .
- إدارة المستخدمين والصلاحيات وفق أدوار مختلفة .
- تمكين المستخدمين من تحميل التذاكر إلكترونياً بصيغة PDF .
- إرسال الإشعارات المتعلقة بالفعاليات والتذاكر .

3.5 القيمة المضافة للنظام

يساهم النظام في تحسين جودة إدارة الفعاليات من خلال:

- تسريع إجراءات الحجز والتسجيل .
- تحسين تجربة المستخدم .
- تقليل الأخطاء التشغيلية .
- تعزيز مستوى الأمان والتحقق من التذاكر .
- توفير بيانات وإحصائيات تساعد متخذي القرار .
- رفع كفاءة إدارة الفعاليات وتقليل الاعتماد على الإجراءات الورقية.

Database Structure & Data Dictionary

Table Name	Column Name	Data Type	Constraints / Foreign Keys	Description
users	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for each user account.
	name	VARCHAR(255)	NOT NULL	The full name of the user.
	email	VARCHAR(255)	Unique, NOT NULL	The email address used for login authentication (must be unique).
	password	VARCHAR(255)	NOT NULL	The securely hashed password of the user.
	role	ENUM	'admin', 'organizer', 'attendee'	The user system role determining access control and dashboard privileges.
	phone	VARCHAR(255)	Nullable	The contact phone number of the user (optional).
events	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for each event.
	organizer_id	BIGINT (Unsigned)	Foreign Key -> users(id)	References the user who organized the event (On Delete Cascade).

	title	VARCHAR(255)	NOT NULL	The title or name of the event.
	description	TEXT	Nullable	A comprehensive description and full details of the event.
	venue	VARCHAR(255)	Nullable	The location or physical/virtual venue where the event takes place.
	start_date	DATETIME	NOT NULL	The scheduled start date and time of the event.
	end_date	DATETIME	Nullable	The scheduled conclusion date and time of the event.
	banner_url	VARCHAR(255)	Nullable	The file path or URL for the event cover/banner image.
	status	ENUM	'pending', 'approved', 'rejected', 'cancelled'	The current operational state of the event registration request.
ticket_types	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for a specific category of ticket.
	event_id	BIGINT (Unsigned)	Foreign Key -> events(id)	References the event to which this ticket tier belongs (On Delete Cascade).
	name	VARCHAR(255)	NOT NULL	The name of the ticket tier (e.g.,

				VIP, General Admission, Early Bird).
	price	DECIMAL(10,2)	NOT NULL	The purchase price for this type of ticket.
	quantity	INT	NOT NULL	The total number of tickets available for purchase in this tier.
	sold	INT	DEFAULT 0	The real-time number of tickets that have already been sold.
orders	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for the complete checkout order transaction.
	user_id	BIGINT (Unsigned)	Foreign Key -> users(id)	References the attendee user who made the purchase (On Delete Cascade).
	event_id	BIGINT (Unsigned)	Foreign Key -> events(id)	References the main event associated with this bulk order (On Delete Cascade).
	total_amount	DECIMAL(10,2)	NOT NULL	The aggregate financial cost of the entire order transaction.
	status	ENUM	'pending', 'paid', 'cancelled', 'refunded'	The financial settlement state of the order placement.

	payment_intent_id	VARCHAR(255)	Nullable	The tracking transaction token from external electronic payment gateways (e.g., Stripe).
order_items	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for a line item within a general order receipt.
	order_id	BIGINT (Unsigned)	Foreign Key -> orders(id)	References the parent order transaction group (On Delete Cascade).
	ticket_type_id	BIGINT (Unsigned)	Foreign Key -> ticket_types(id)	References the reserved ticket tier (On Delete Cascade).
	quantity	INT	NOT NULL	The precise amount of tickets requested for this ticket tier.
	price	DECIMAL(10,2)	NOT NULL	The locked historical unit price at the exact moment of order placement.
tickets	id	BIGINT (Unsigned)	Primary Key, Auto Increment	The unique identifier for an individually issued ticket.
	ticket_type_id	BIGINT (Unsigned)	Foreign Key -> ticket_types(id)	References the associated ticket tier structure (On Delete Cascade).

	attendee_id	BIGINT (Unsigned)	Foreign Key -> users(id)	References the concrete user holding this ticket item (On Delete Cascade).
	order_id	BIGINT (Unsigned)	Foreign Key -> orders(id)	References the checkout transaction that produced this ticket (On Delete Cascade).
	qr_code	LONGTEXT	Unique (Hash)	A uniquely hashed alphanumeric string encoded into a QR code for security verification at the gate.
	status	ENUM	'valid', 'used', 'cancelled'	The physical entry validation state of the ticket copy.
notifications	id	CHAR(36)	Primary Key (UUID)	The unique standard UUID string for each broadcast notification instance.
	type	VARCHAR(255)	NOT NULL	The Laravel application routing notification handler class path (e.g., OrderCreated).
	notifiable_type	VARCHAR(255)	NOT NULL	Polymorphic target relation model path maps to entities like 'App\Models\User'.
	notifiable_id	BIGINT (Unsigned)	NOT NULL	The relative row ID value of the model class

				targeted by this notification item.
	data	LONGTEXT	JSON Format, NOT NULL	A JSON string containing the payload object context (messages, URLs, transaction specs).
	read_at	TIMESTAMP	Nullable	The exact timestamp when the user marked the interface notification as read.

Laravel Framework System Tables

Table Name	Functional Description & Operational Role
migrations	Tracks and monitors data migration batch files executed in the system schema to maintain structure version control.
cache	Stores short-term application cache metrics to boost the rendering and loading speed of the system.
cache_locks	Acts as an operation mutex layer ensuring parallel tasks do not corrupt transactional state limits.
jobs	Manages the execution queue pipeline for heavier asynchronous operations (e.g., broadcast emails).
failed_jobs	Maintains an incident trace log for background tasks or queue workers that failed to finish successfully.

job_batches	Coordinates and groups multiple asynchronous tracking jobs executing parallel structural logic stacks.
sessions	Stores connection states, IP tracking tokens, user device agents, and persistence for live platform clients.
password_reset_tokens	Stores individual authorization hashes linked to account password recovery pipelines.
personal_access_tokens	Tracks system-wide secure bearer token access configurations via Laravel Sanctum API filters.